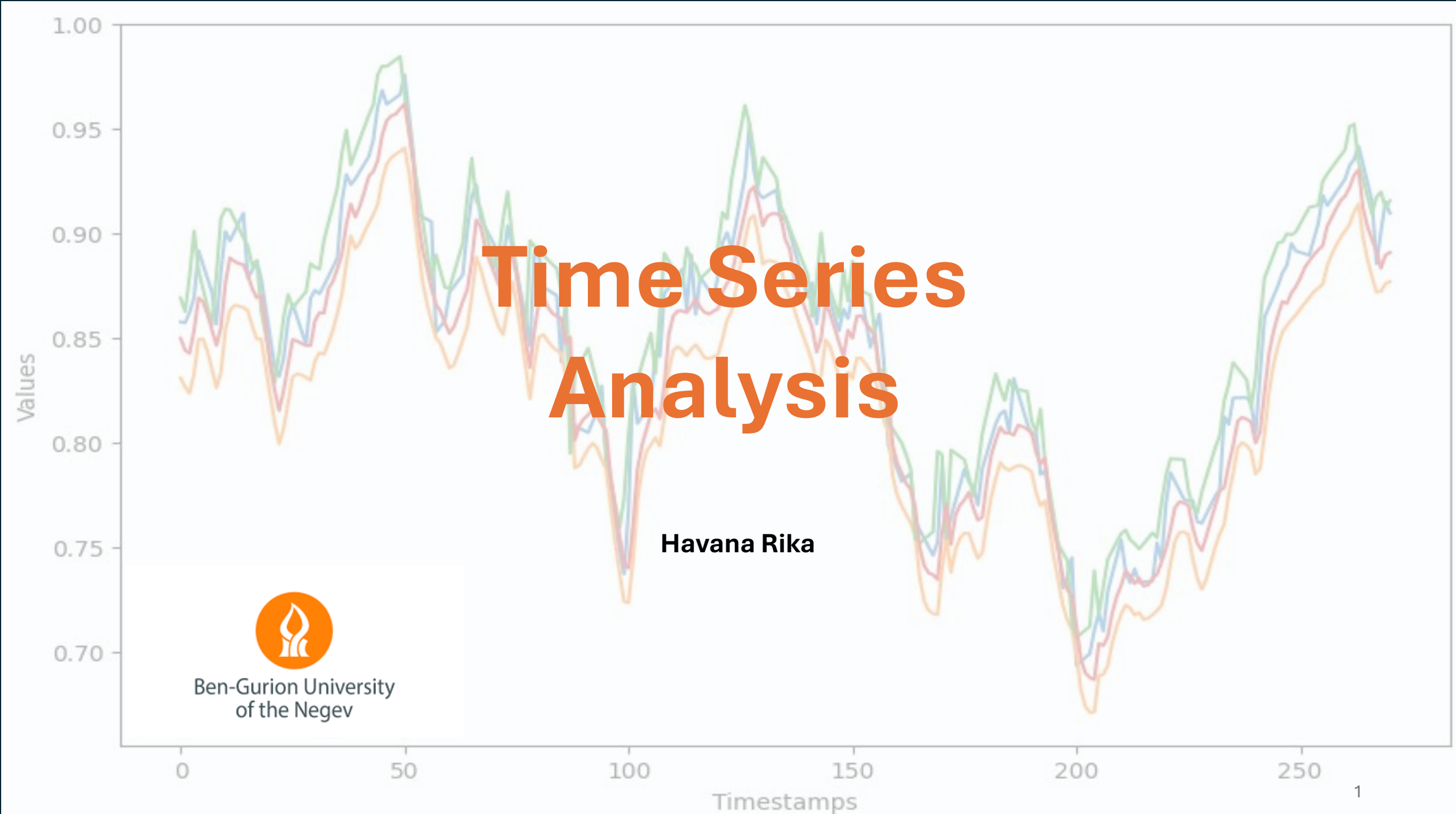


Time Series Analysis

Havana Rika



Ben-Gurion University
of the Negev



Pandas Datetime Operations

An easy, well-formatted **stock exchange price** dataset from the UCI ML Repository

```
# Skipping first row cause it doesn't have any data  
df = pd.read_excel("https://archive.ics.uci.edu/ml/machine-learning-  
databases/00247/data_akbilgic.xlsx", skiprows=1)
```

	date	ISE	ISE.1	SP	DAX	FTSE	NIKKEI	BOVESPA	EU	EM
0	2009-01-05	0.035754	0.038376	-0.004679	0.002193	0.003894	0.000000	0.031190	0.012698	0.028524
1	2009-01-06	0.025426	0.031813	0.007787	0.008455	0.012866	0.004162	0.018920	0.011341	0.008773
2	2009-01-07	-0.028862	-0.026353	-0.030469	-0.017833	-0.028735	0.017293	-0.035899	-0.017073	-0.020015
3	2009-01-08	-0.062208	-0.084716	0.003391	-0.011726	-0.000466	-0.040061	0.028283	-0.005561	-0.019424
4	2009-01-09	0.009860	0.009658	-0.021533	-0.019873	-0.012710	-0.004474	-0.009764	-0.010989	-0.007802

Converting str into pd.Timestamp/DatetimeIndex

First, we must convert the date column (which may not always be parsed as dates automatically by pandas) into **pandas datetime format**.

The function **pd.to_datetime** converts the input into a **pd.Timestamp**, if the input is a **string**, or into a **DatetimeIndex**, if the input is a **list of strings**.

We will also use **strftime** function, which formats the date representation into a format we specify. It uses strftime conventions to specify the format of the data. For instance, %d means a zero-padded date, %B means a month's full name, and %Y means a year in four digits.

A full list of strftime conventions can be found at <https://docs.python.org/3/library/datetime.html>

```
>>> pd.to_datetime("13-4-1987").strftime("%d, %B %Y")  
'13, April 1987'
```

Converting str into pd.Timestamp

In the following case, the automatic parsing fails since the date is January 4, 1987.

```
>>> pd.to_datetime("4-1-1987").strftime("%d, %B %Y")  
'01, April 1987'
```

Pandas assumes the month comes first; we need to specify whether the month or the day comes first.

```
>>> pd.to_datetime("4-1-1987", dayfirst=True).strftime("%d, %B %Y")  
'04, January 1987'
```

Another case where automatic date parsing fails is when the date string is in a non-standard form. In that case, we can provide a strftime-formatted string to help pandas parse the dates correctly.

```
>>> pd.to_datetime("4|1|1987", format="%d|%m|%Y").strftime("%d, %B %Y")  
'04, January 1987'
```

Converting str into pd.Timestamp

convert the date column in our stock prices dataset into datetime

```
df['date'] = pd.to_datetime(df['date'], yearfirst=True)
```

Now, the 'date' column, dtype, should be either datetime64[ns] or <M8[ns], which are both pandas & NumPy-native datetime formats, which **unlock** a **wide range** of additional **functionalities**.

```
>>> df.date.min(),df.date.max()  
(Timestamp('2009-01-05 00:00:00'), Timestamp('2011-02-22 00:00:00'))
```

Using the datetime properties

We can access many **datetime properties**, such as month, day_of_week, day_of_year, and so on, using the **.dt accessor**

```
>>> print(f"""
    Date: {df.date.iloc[0]}
    Day of year: {df.date.dt.day_of_year.iloc[0]}
    Day of week: {df.date.dt.dayofweek.iloc[0]}
    Month: {df.date.dt.month.iloc[0]}
    Month Name: {df.date.dt.month_name().iloc[0]}
    Quarter: {df.date.dt.quarter.iloc[0]}
    Year: {df.date.dt.year.iloc[0]}
    ISO Week: {df.date.dt.isocalendar().week.iloc[0]}
    """)

Date: 2009-01-05 00:00:00
Day of year: 5
Day of week: 0
Month: 1
Month Name: January
Quarter: 1
Year: 2009
ISO Week: 2
```

Indexing and slicing

when we make the **date** column the **index** of the DataFrame, we can use all the **slicing operations** that pandas supports but **on the datetime** axis.

```
# Setting the index as the datetime column  
df.set_index("date", inplace=True)  
# Select all data after 2010-01-04(inclusive)
```

```
df["2010-01-04":]  
# Select all data between 2010-01-04 and 2010-02-06(exclusive)  
df["2010-01-04": "2010-02-06"]  
# Select data 2010 and before  
df[: "2010"]  
# Select data between 2010-01 and 2010-06(both including)  
df["2010-01": "2010-06"]
```

Create a Single date

Datetime module in python —
Basic date and time types

The datetime module supplies classes for **manipulating dates and times**. While date and time arithmetic is supported, the focus of the implementation is on efficient **attribute extraction** for output formatting and manipulation.

```
datetime.datetime(2012, 5, 1)
```

May 1, 2012 at 00:00:00

datetime objects

A [datetime](#) object is a single object containing all the information from a [date](#) object and a [time](#) object.

Like a [date](#) object, [datetime](#) assumes the current Gregorian calendar extended in both directions; like a [time](#) object, [datetime](#) assumes there are exactly 3600*24 seconds in every day.

Constructor:

```
class datetime.datetime(year, month, day, hour=0, minute=0, second=0, microsecond=0, tzinfo=None, *, fold=0)
```

The *year*, *month* and *day* arguments are required. *tzinfo* may be `None`, or an instance of a [tzinfo](#) subclass.

The remaining arguments must be integers in the following ranges:

- `MINYEAR <= year <= MAXYEAR`,
- `1 <= month <= 12`,
- `1 <= day <= number of days in the given month and year`,
- `0 <= hour < 24`,
- `0 <= minute < 60`,
- `0 <= second < 60`,
- `0 <= microsecond < 1000000`,
- `fold` in `[0, 1]`.

If an argument outside those ranges is given, [ValueError](#) is raised.

Creating date sequences

pandas has something similar to range for datetime—**pd.date_range**.

The function accepts **start** and **end** dates, along with a **frequency** (daily, monthly, and so on), and creates the **sequence of dates in between**.

```
# Specifying start and end dates with frequency
pd.date_range(start="2018-01-20", end="2018-01-23", freq="D").astype(str).tolist()
# Output: ['2018-01-20', '2018-01-21', '2018-01-22', '2018-01-23']
# Specifying start and number of periods to generate in the given frequency
pd.date_range(start="2018-01-20", periods=4, freq="D").astype(str).tolist()
# Output: ['2018-01-20', '2018-01-21', '2018-01-22', '2018-01-23']
# Generating a date sequence with every 2 days
pd.date_range(start="2018-01-20", periods=4, freq="2D").astype(str).tolist()
# Output: ['2018-01-20', '2018-01-22', '2018-01-24', '2018-01-26']
# Generating a date sequence every month. By default it starts with Month end
pd.date_range(start="2018-01-20", periods=4, freq="M").astype(str).tolist()
# Output: ['2018-01-31', '2018-02-28', '2018-03-31', '2018-04-30']
# Generating a date sequence every month, but month start
pd.date_range(start="2018-01-20", periods=4, freq="MS").astype(str).tolist()
# Output: ['2018-02-01', '2018-03-01', '2018-04-01', '2018-05-01']
```

Creating date sequences

```
In [82]: start = datetime.datetime(2011, 1, 1)

In [83]: end = datetime.datetime(2012, 1, 1)

In [84]: index = pd.date_range(start, end)

In [85]: index
Out[85]:
DatetimeIndex(['2011-01-01', '2011-01-02', '2011-01-03', '2011-01-04',
               '2011-01-05', '2011-01-06', '2011-01-07', '2011-01-08',
               '2011-01-09', '2011-01-10',
               ...,
               '2011-12-23', '2011-12-24', '2011-12-25', '2011-12-26',
               '2011-12-27', '2011-12-28', '2011-12-29', '2011-12-30',
               '2011-12-31', '2012-01-01'],
              dtype='datetime64[us]', length=366, freq='D')

In [86]: index = pd.bdate_range(start, end)
```

Managing date offsets

We can also add or subtract days, months, and other values to/from dates using `pd.TimeDelta`

```
# Add four days to the date range
(pd.date_range(start="2018-01-20", end="2018-01-23", freq="D") +
pd.Timedelta(4, unit="D")).astype(str).tolist()
# Output: ['2018-01-24', '2018-01-25', '2018-01-26', '2018-01-27']

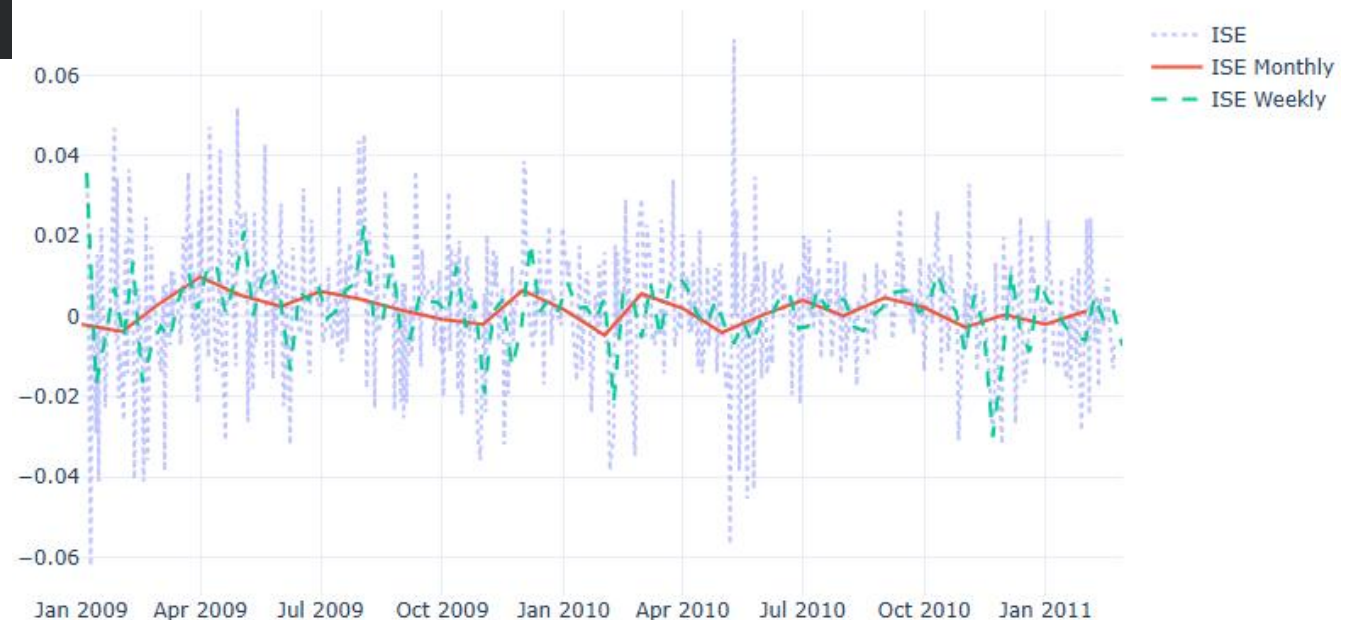
# Add four weeks to the date range
(pd.date_range(start="2018-01-20", end="2018-01-23", freq="D") +
pd.Timedelta(4, unit="W")).astype(str).tolist()
# Output: ['2018-02-17', '2018-02-18', '2018-02-19', '2018-02-20']
```

https://pandas.pydata.org/docs/user_guide/timeseries.html#timeseries-offset-aliases

Resampling Operation

Resampling is a powerful operation which let's you aggregate and disaggregate along the temporal dimension.

```
# Resampling at Month Start and taking the mean
m_df = df[['ISE']].resample("MS").mean()
display(m_df.head())
# Resampling at Week(starting Monday) and taking the mean
w_df = df[['ISE']].resample("W-MON").mean()
display(w_df.head())
```

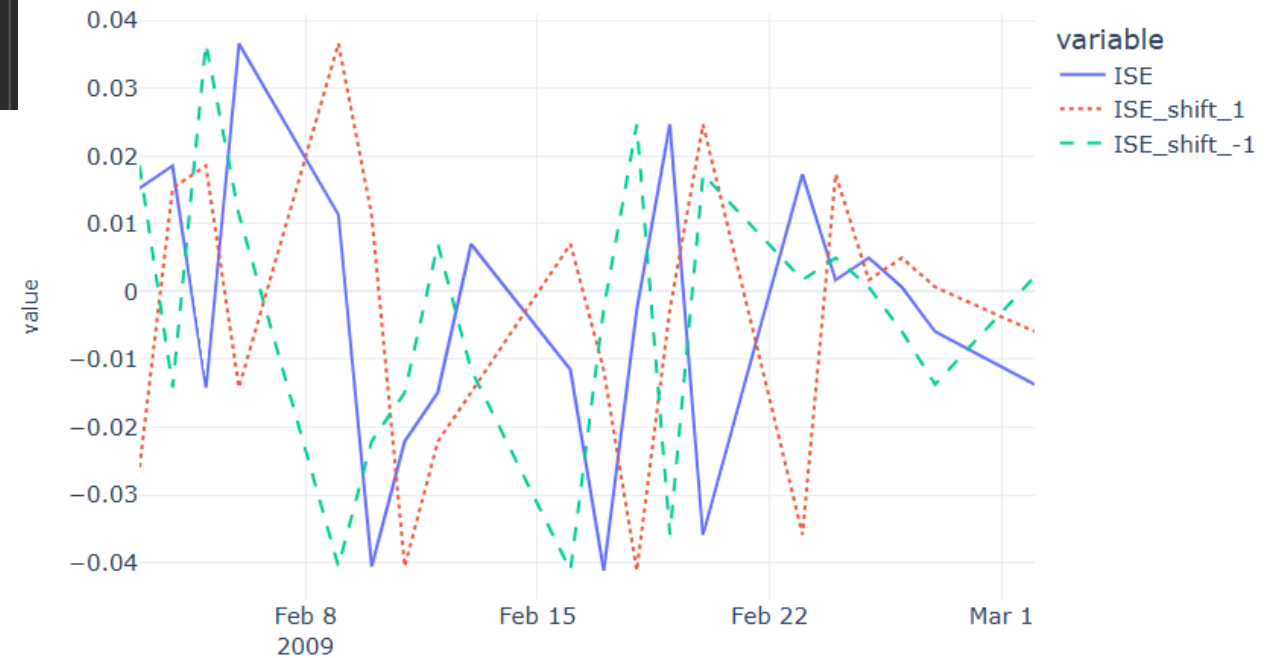


Shifting Operation

```
plot_df = df.loc[:, ['ISE']]
# Shift forward by one day
plot_df['ISE_shift_1'] = df[['ISE']].shift(1)
# Shift backward by one day
plot_df['ISE_shift_-1'] = df[['ISE']].shift(-1)

plot_df = plot_df.dropna().iloc[20:40].reset_index()
plot_df = pd.melt(plot_df, id_vars="date")
fig = px.line(plot_df, x="date", y="value", color="variable", line_dash="variable", template="plotly_white")
fig.update_layout(
    yaxis=dict(tickfont=dict(size=15)),
    xaxis=dict(tickfont=dict(size=15)),
    legend=dict(font=dict(size=15)),
)
fig.show()
```

Shifting a series along the temporal dimension is another valuable operation

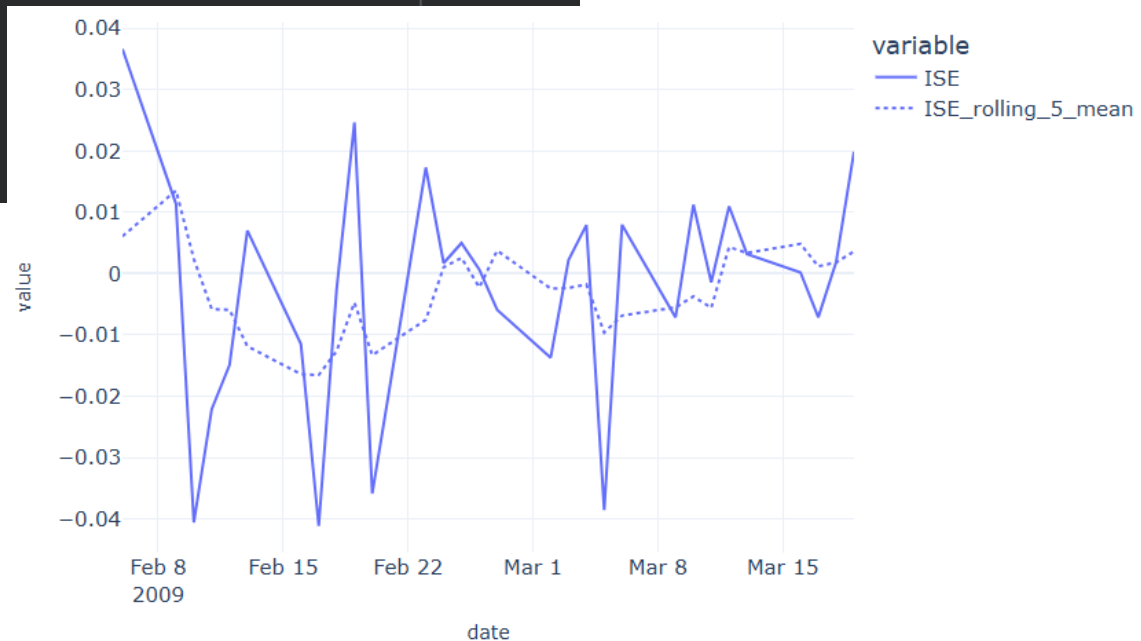


Rolling Window Operation

There are strong use cases to calculate some statistics on a rolling or expanding window.

```
plot_df = df.loc[:, ['ISE']]
# Moving Average of 5 days
plot_df["ISE_rolling_5_mean"] = df[['ISE']].rolling(window=5).mean()

plot_df = plot_df.dropna().iloc[20:50].reset_index()
plot_df = pd.melt(plot_df, id_vars="date")
fig = px.line(plot_df, x="date", y="value", line_dash="variable", template="plotly_white")
fig.update_layout(
    yaxis=dict(tickfont=dict(size=15)),
    xaxis=dict(tickfont=dict(size=15)),
    legend=dict(font=dict(size=15)),
)
fig.show()
```



Expanding Window Operation

```
plot_df = df.loc[:, ['ISE']]
# Moving Average of 30 days
plot_df["ISE_rolling_30_mean"] = df[['ISE']].rolling(window=120).mean()
# Expanding Average
plot_df["ISE_expanding_mean"] = df[['ISE']].expanding().mean()
plot_df.drop(columns='ISE', inplace=True)
plot_df = plot_df.dropna().reset_index()
plot_df = pd.melt(plot_df, id_vars="date")
fig = px.line(plot_df, x="date", y="value", line_dash="variable", template="plotly_white")
fig.update_layout(
    yaxis=dict(tickfont=dict(size=15)),
    xaxis=dict(tickfont=dict(size=15)),
    legend=dict(font=dict(size=15)),
)
fig.show()
```



Main difference

Resampling - Changes the **time frequency**.

Shifting - Moves values to create **lags/leads**.

Rolling window - Looks at a **fixed recent window**.

Expanding window - Looks at **all past data so far**.

Handling missing data

When working with large datasets, **missing values** should not be handled automatically by dropping rows or filling with the mean.

First, consider the **data-generating process**. Sometimes what looks like missing data actually carries meaning. For example, if a supermarket product has no transactions on a day, that usually means **zero sales**, not missing data, so those gaps should be filled with zero.

If the missingness follows a **pattern**, such as data being absent every Sunday, handling it depends on the model you plan to use. Filling those gaps with zeros can mislead some forecasting models unless you also include information like the day of the week so the model can recognize the pattern.

Finally, if a product that normally sells well suddenly shows **zero sales**, that may indicate a real data problem or special event, such as a point-of-sale (POS) failure, entry error, or out-of-stock situation. In those cases, the values may need to be **imputed carefully** using appropriate techniques.

The Air Quality dataset

The Air Quality dataset **published by the ACT Government**, Canberra, Australia, under the CC by Attribution 4.0 International License (<https://www.data.act.gov.au/Environment/Air-Quality-Monitoring-Data/94a5-zqnn>) and see how we can **impute** such **values** using **pandas**.

```
df = pd.read_csv("au_airquality.csv", sep=",")
df.head()
```

	name	gps	datetime	no2	o3_1hr	o3_4hr	o3_8hr	co	pm10_1_hr	pm2_5_1_hr	...	aqi_cc
0	Monash	\n, \n(-35.418302, 149.094018)	2024-03-18T16:00:00.000	NaN	NaN	NaN	0.019	NaN	8.5	2.1	...	NaN
1	Monash	\n, \n(-35.418302, 149.094018)	2024-02-22T16:00:00.000	NaN	NaN	NaN	0.024	NaN	0.0	0.0	...	NaN
2	Monash	\n, \n(-35.418302, 149.094018)	2024-02-15T16:00:00.000	NaN	NaN	NaN	0.016	NaN	9.4	NaN	...	NaN
3	Monash	\n, \n(-35.418302, 149.094018)	2024-02-10T16:00:00.000	NaN	NaN	NaN	0.018	NaN	5.4	4.1	...	NaN
4	Florey	\n, \n(-35.220606, 149.043539)	2024-01-15T16:00:00.000	NaN	NaN	NaN	0.017	NaN	2.6	1.4	...	NaN

5 rows x 22 columns

The Air Quality dataset

```
#Selecting one location and pm2.5
df = df.loc[df.name=="Monash", ['datetime', 'pm2_5_1_hr']]
df.dtypes
```

```
0
datetime    object
pm2_5_1_hr  float64
dtype: object
```

```
df.datetime = pd.to_datetime(df.datetime)
df.sort_values("datetime", inplace=True)
df.set_index("datetime", inplace=True)
df.head()
```

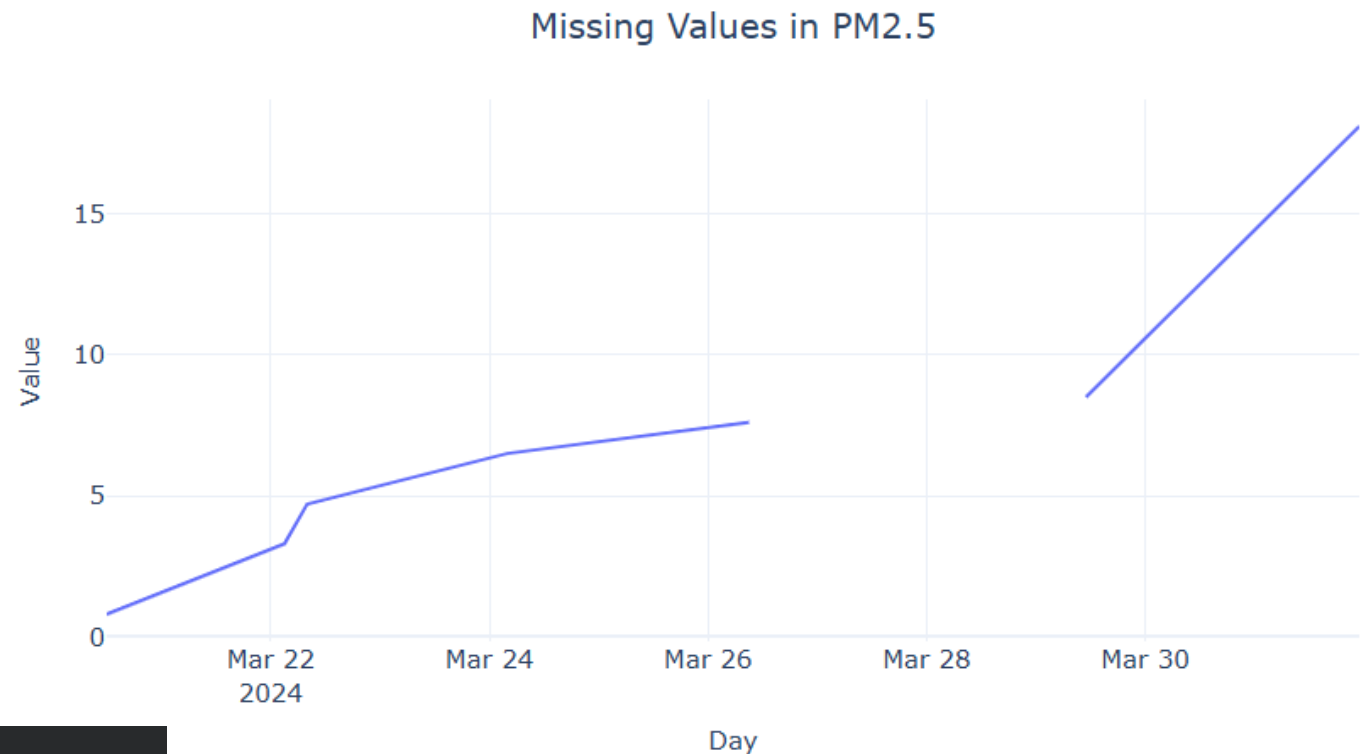
	pm2_5_1_hr
datetime	
2012-08-31 16:00:00	NaN
2013-03-18 16:00:00	NaN
2014-02-24 21:00:00	NaN
2014-04-24 16:00:00	NaN
2015-01-28 05:00:00	NaN

Artificial missing values

We have chosen the region **Monash** and **PM2.5** readings, and artificially introduced some missing values, as shown in the following diagram

```
dates = df.index.unique().tolist()
null_dates = dates[-10:-7]
# introducing missing values by nulling out the values
df.loc[null_dates, "pm2_5_1_hr"] = np.nan
```

```
df = df.loc[dates[-15:-5], "pm2_5_1_hr"].to_frame()
fig = px.line(df, x=df.index, y="pm2_5_1_hr", title="Missing Values in PM2.5")
fig = format_plot(fig, ["Original"])
fig.show()
```



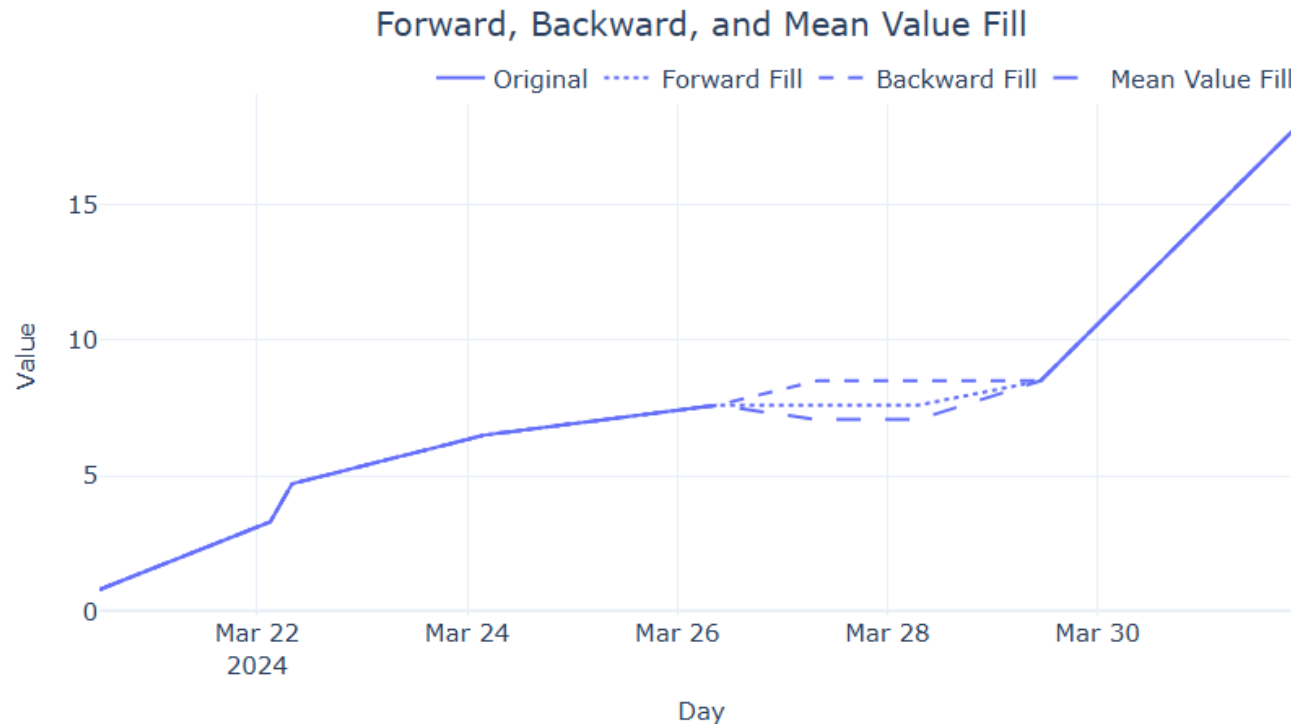
Techniques to fill the missing values

- **Last Observation Carried Forward or Forward Fill:** This imputation technique takes the last observed value, using that to fill in all the missing values until it finds the next observation. This is also called forward fill.
- **Next Observation Carried Backward or Backward Fill:** This imputation technique takes the next observation and backtracks to fill in all the missing values with this value. This is also called backward fill.
- **Mean Value Fill:** This imputation technique is also pretty simple. We calculate the mean of the entire series, and wherever we find missing values, we fill it with the mean value.

```
_df = df.copy()
#Forward Fill
_df["ffill"] = _df['pm2_5_1_hr'].ffill()
# Backward Fill
_df["bfill"] = _df['pm2_5_1_hr'].bfill()
# Mean Value Fill
_df["mean_value"] = _df['pm2_5_1_hr'].fillna(df['pm2_5_1_hr'].mean())
```

Forward, Backward, and Mean Value Fill

```
#Plotting
plot_df = pd.melt(_df.reset_index(), id_vars="datetime", var_name="series")
fig = px.line(plot_df, x="datetime", y=["value"], line_dash="series", title="Forward, Backward, and Mean Value Fill")
fig = format_plot(fig, ['Original', 'Forward Fill', 'Backward Fill', "Mean Value Fill"])
fig.show()
```



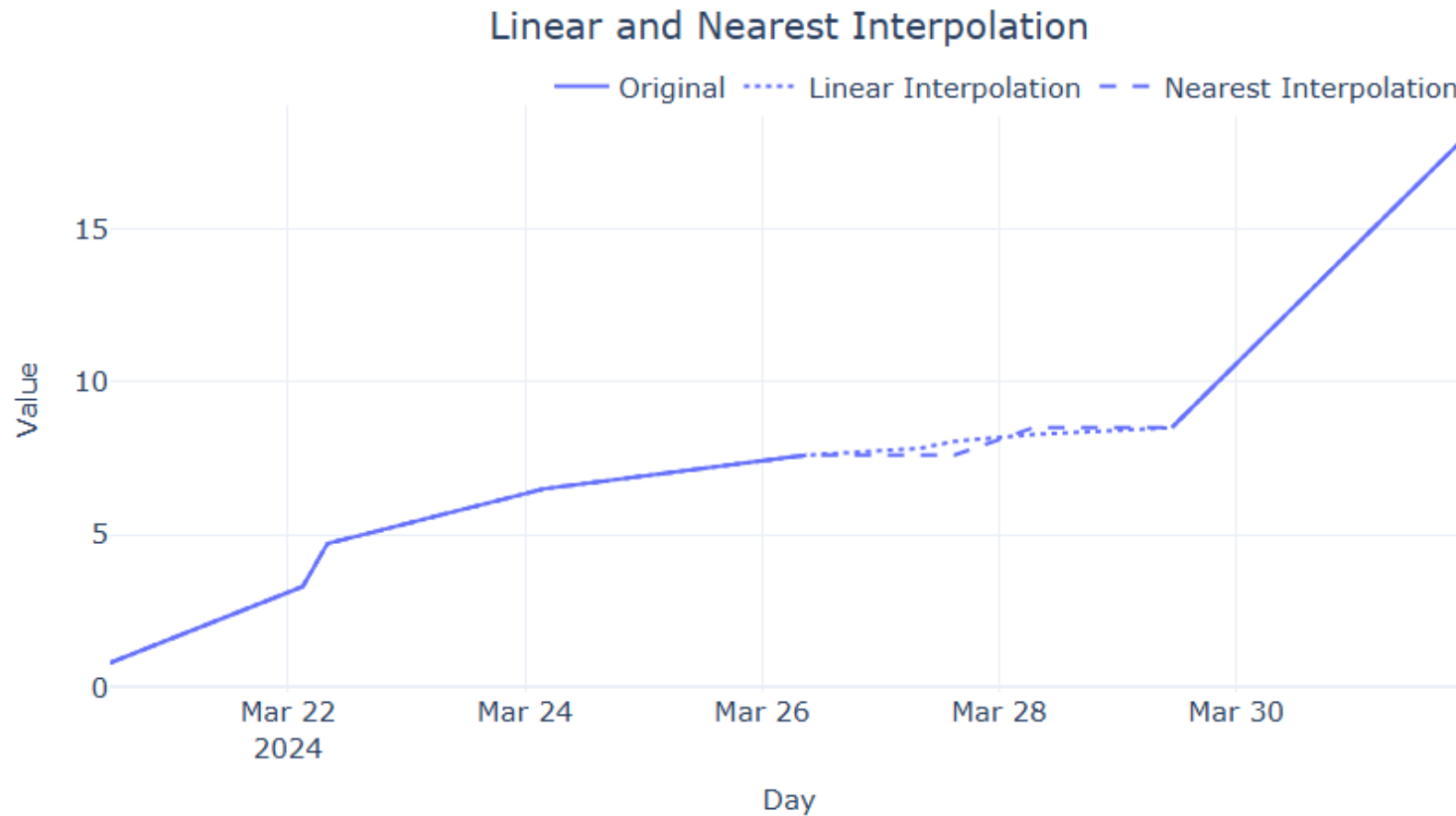
Linear and Nearest Interpolation

- **Linear Interpolation:** Linear interpolation is just like drawing a line between the two observed points and filling in the missing values so that they lie on this line.
- **Nearest Interpolation:** This is intuitively like a combination of the forward and backward fill. For each missing value, the closest observed value is found and used to fill it in.

```
_df = df.copy()
_df["linear_interpolation"] = _df['pm2_5_1_hr'].interpolate(method="linear")
_df["nearest_interpolation"] = _df['pm2_5_1_hr'].interpolate(method="nearest")
```

Linear and Nearest Interpolation

```
#Plotting
plot_df = pd.melt(_df.reset_index(), id_vars="datetime", var_name="series")
fig = px.line(plot_df, x="datetime", y=["value"], line_dash="series", title="Linear and Nearest Interpolation")
fig = format_plot(fig, ['Original', "Linear Interpolation", "Nearest Interpolation"])
fig.show()
```



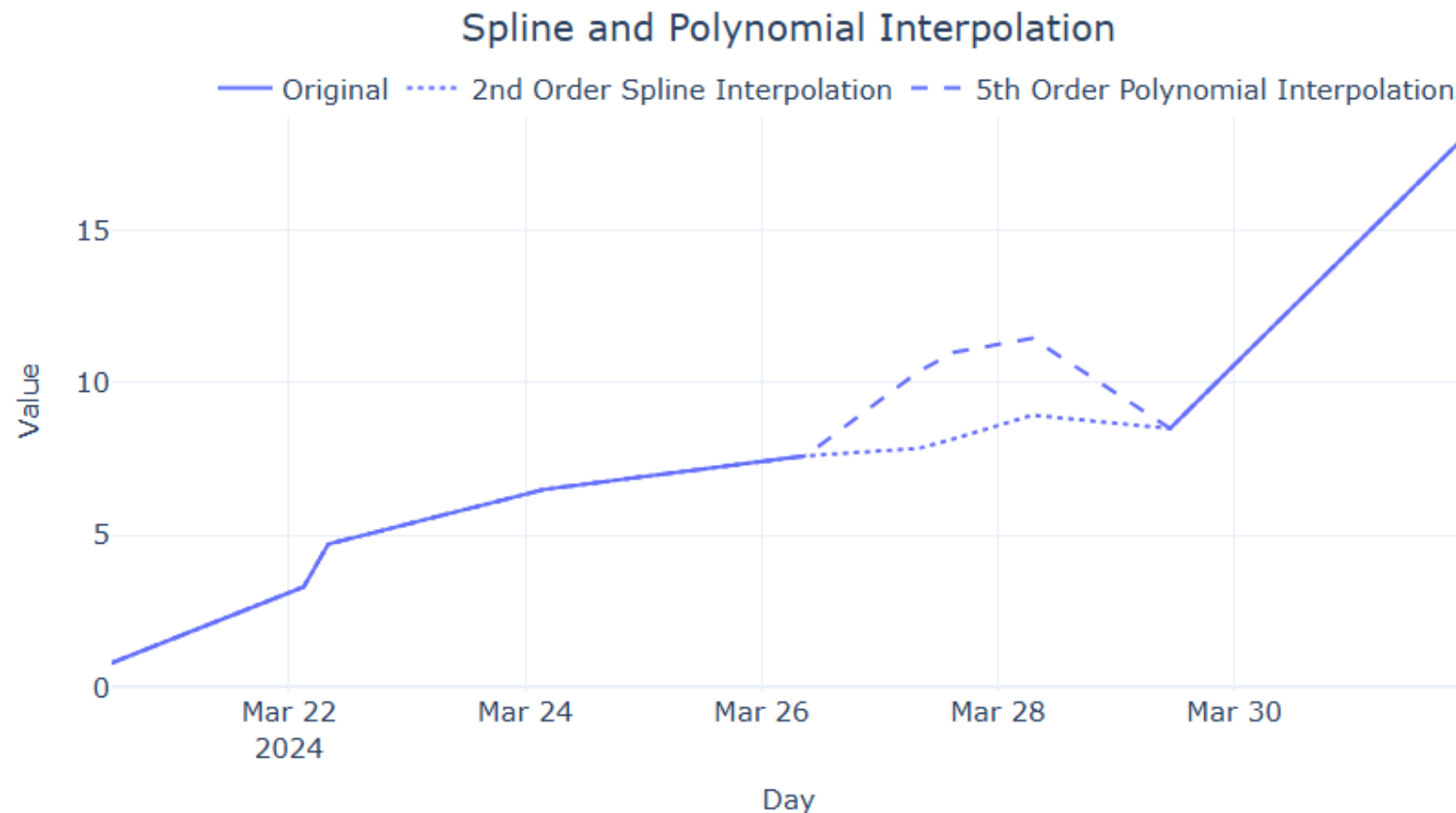
Non-linear Interpolation

- **Spline, Polynomial, and Other Interpolations:** In addition to linear interpolation, pandas also supports non-linear interpolation techniques that call a SciPy routine at the backend.
- **Spline** and **polynomial** interpolations are **similar**. They fit a spline/polynomial of a given order to the data and use that to fill in missing values. While using spline or polynomial as the method in interpolate, **we should always provide order** as well.
- The higher the order, the more flexible the function that is used to fit the observed points will be.

```
_df = df.copy()
_df["spline_interpolation"] = _df['pm2_5_1_hr'].interpolate(method="spline", order=2)
_df["polynomial_interpolation"] = _df['pm2_5_1_hr'].interpolate(method="polynomial", order=5)
```

Spline and Polynomial Interpolation

```
#Plotting
plot_df = pd.melt(_df.reset_index(), id_vars="datetime", var_name="series")
fig = px.line(plot_df, x="datetime", y=["value"], line_dash="series", title="Spline and Polynomial Interpolation")
fig = format_plot(fig, ['Original', "2nd Order Spline Interpolation", "5th Order Polynomial Interpolation"])
fig.show()
```



Smart meters in London

The dataset contains **energy consumption** readings for a sample of **5,567 London households** that took part in the UK Power Networks-led Low Carbon London project between November 2011 and February 2014. Readings were taken at **half-hourly intervals**.

The available metadata:

- CACI UK segmented the UK's population into **demographic types**, called Acorn.
- The dataset contains two groups of customers. One group who was subjected to **dynamic timeof-use (dToU)** energy prices throughout 2013, and another group who were on **flat-rate tariffs**. The tariff prices for the dToU were given **a day ahead**, via the smart meter IHD or text message.
- Jean-Michel D also enriched the dataset with **weather** and **UK bank holiday** data.

<https://www.kaggle.com/datasets/jeanmidev/smart-meters-in-london>

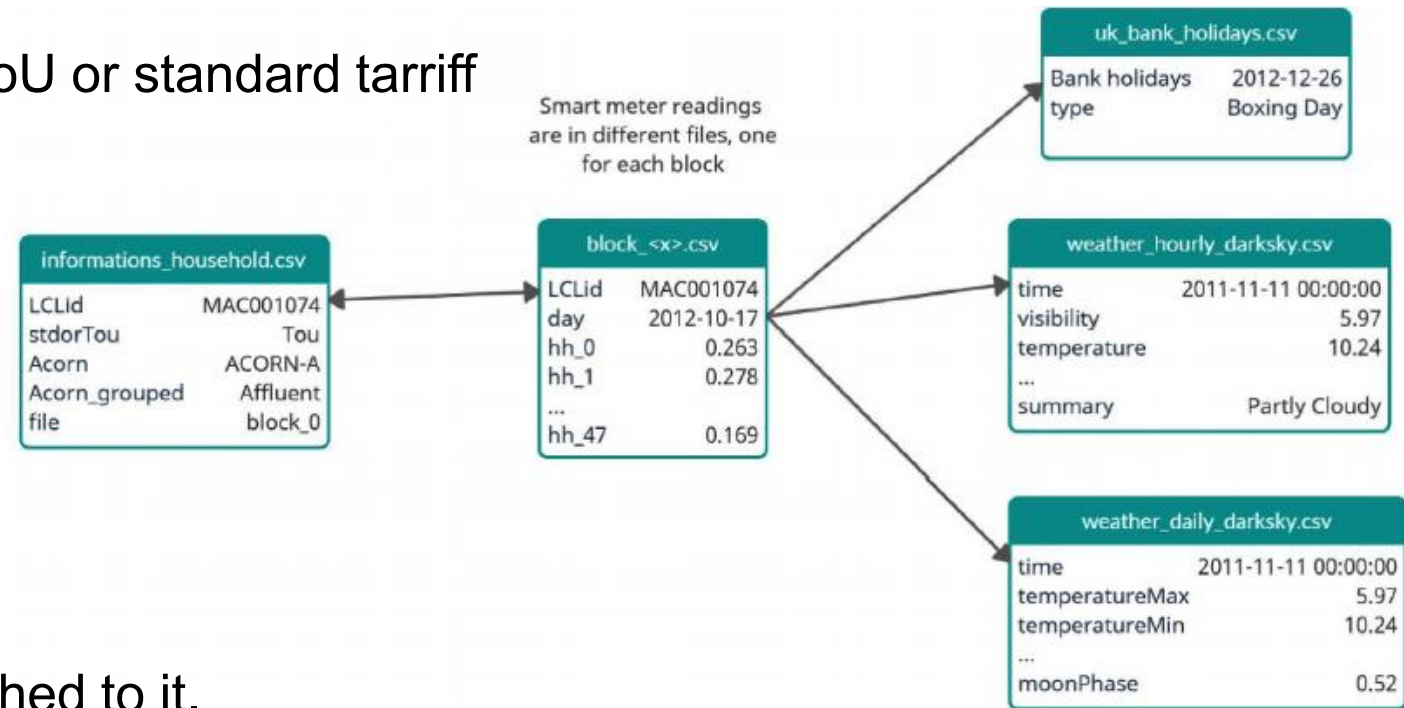
The Acorn Classes

Acorn Group	Acorn Class
Affluent Achievers	A-Lavish Lifestyles
	B-Executive Wealth
	C-Mature Money
Rising Prosperity	D-City Sophisticates
	E-Career Climbers
Comfortable Communities	F-Countryside Communities
	G-Successful Suburbs
	H-Steady Neighborhoods
	I-Comfortable Seniors
	J-Starting Out
Financially Stretched	K-Student Life
	L-Modest Means
	M-Striving Families
	N-Poorer Pensioners
Urban Adversity	O-Young Hardship
	P-Struggling Estates
	Q-Difficult Circumstances

Data Model of the London Smart Meters Dataset

Let's look at a few key column names and their meanings:

- **LCLid**: The unique consumer ID for a household
- **stdorTou**: Whether the household has dToU or standard tariff
- **Acorn**: The ACORN class
- **Acorn_grouped**: The ACORN group
- **file**: The block number



Each **LCLid** has a **unique** time series attached to it.

The time series file is formatted in a slightly tricky format - **each day, there will be 48 observations at a half-hourly frequency** in the columns of the file.

Converting the half-hourly into TS data

A few general categories of information we will find in a time series dataset:

- **Time Series Identifiers:** These are identifiers for a particular time series. It can be a name, an ID, or any other unique feature - for example, the SKU name or the ID of a retail sales dataset or the consumer ID in the energy dataset that we are working with are all time series identifiers.
- **Metadata or Static Features:** This information does not vary with time. An example of this is the ACORN classification of the household in our dataset.
- **Time-Varying Features:** This information varies with time - for example, the weather information.

For each point in time, we have a different value for weather, unlike the Acorn classification.

Compact, expanded, and wide forms of data

Time series datasets can be organized in several formats. The traditional one is **wide format**, where the date acts like an index and **each separate time series gets its own column**.

This becomes **impractical** when there are many series, because the table grows very wide and cannot easily store metadata about each series.

This data format **does not allow us to include any metadata** about the time series.

From an operational perspective, the wide format also **does not play well with relational databases** because we have to keep adding columns to a table when we get new time series.

We won't be using this format in this book.

Compact, expanded, and wide forms of data

In **compact format**, each entire time series is stored in a single row, with metadata in normal columns and the time-varying values stored as arrays. **Extra columns like start_datetime and frequency are used to reconstruct the timeline.** This format is memory-efficient and works well for regularly sampled data.

LCLid	start_datetime	frequency	energy_consumption	series_length
MAC000002	2012-10-13	30min	[0.263, 0.26899999999999999, 0.275, 0.256, 0.21...	24144
MAC000246	2011-12-04	30min	[0.175, 0.098, 0.144, 0.065, 0.071, 0.037, 0.0...	39216
MAC000450	2012-03-23	30min	[0.337, 1.426, 0.996, 0.971, 0.994, 0.952, 0.8...	33936
MAC001074	2012-05-09	30min	[0.18, 0.086, 0.106, 0.173, 0.146, 0.223, 0.21...	31680
MAC003223	2012-09-18	30min	[0.076, 0.079, 0.123, 0.109, 0.051, 0.069, 0.0...	25344

Compact, expanded, and wide forms of data

In **expanded format** (also called **long format**), each time step of a series gets its own row. The identifiers and metadata are repeated across rows, and there is an explicit timestamp column. This format **is more flexible** and **works better for handling metadata and database operations**.

LCLid	energy_consumption	series_length	timestamp	frequency
MAC000002	0.263	24144	2012-10-13 00:00:00	30min
MAC000002	0.269	24144	2012-10-13 00:30:00	30min
MAC000002	0.275	24144	2012-10-13 01:00:00	30min
MAC000002	0.256	24144	2012-10-13 01:30:00	30min
MAC000002	0.211	24144	2012-10-13 02:00:00	30min

Enforcing regular intervals in time series

One of the first things you should check and correct is whether the regularly sampled time series data that you have has **equal intervals of time**.

In practice, even regularly sampled time series have some samples missing in between, due to some data collection error or some other peculiar way data is collected.

So while working with the data, we will make sure we enforce regular intervals in the time series.

In our smart meters dataset, some **LCLid** columns end much earlier than the rest. Maybe the household opted out of the program, or they moved out and left the house empty; the reason could be anything.

However, we need to handle that while we enforce regular intervals.

Convert the London Smart Meters dataset to TS

For each dataset that you come across, the steps you would have to take to convert it into either a compact or expanded form would be different. It depends on how the original data is structured.

We will look at how the London Smart Meters dataset can be transformed.

There are two steps we need to do before we can start processing the data into either compact or expanded form:

1. **Find the Global End Date:** We must find the maximum date across all the block files so that we know the global end date of the time series.
2. **Basic Preprocessing:** If you remember how `hhblock_dataset` is structured, you will remember that each row has a date and that, along the columns, we have half-hourly blocks. We need to reshape that into a long form, where each row has a date and a single half-hourly block. It's easier to handle that way.

Expanded form

The function to convert data into the expanded form does the following:

1. Finds the **start date**.
2. **Create** a standard **DataFrame** using the start date and the global end date.
3. **Left-merges the DataFrame** for LCLid with the standard DataFrame, leaving the missing data as np.nan.
4. Returns the merged DataFrame.

Once we have all the LCLid DataFrames, we must perform a couple of additional steps to complete the expanded form processing:

1. **Concatenate** all the DataFrames into a single DataFrame.
2. **Create a column** called offset, which is the numerical representation of the half-hour blocks.
3. **Create a timestamp** by adding a 30-minute offset to the day and dropping the unnecessary columns.

Compact form

The function for converting into compact form does the following:

1. **Finds the start date** and time series identifiers.
2. **Creates** a standard **DataFrame** using the start date and the global end date.
3. **Left merges the DataFrame** for LCLid to the standard DataFrame, leaving the missing data as np.nan.
4. **Sorts** the values on the **date**.
5. **Returns** the time series **array**, along with the time series **identifier**, **start date**, and the length of the time series.

Once we have this information for each LCLid, we can compile it into a DataFrame and add **30min** as the frequency.

For one block, this representation takes up only **~0.002 MB of memory. Vs. ~47 MB of memory** in the Expanded form.

We are going to use the compact form because it is easy to work with and much less resource-hungry.

Mapping additional information

From the data model that we prepared earlier, we know that there are three key files that we have to map: **Household** Information, **Weather**, and **Bank Holidays**.

Household information is static metadata, so it can be added with a simple left join on LCLid, the time series identifier.

Bank holidays and **weather** are different because they change over time, so they must be aligned exactly with the time series arrays.

Mapping additional information

For **bank holidays**, the date column is converted to datetime, set as the index, resampled to **30-minute intervals**, then forward-filled within each day, with remaining missing values filled as **NO_HOLIDAY**. This creates a holiday label for every half-hour timestamp.

For **weather**, the same general process is used: convert the date column to datetime, set it as the index, resample to **30-minute frequency**, and forward-fill the weather values so that every half-hour timestamp has matching weather data. Once both time-varying datasets are aligned to the same frequency as the time series, their values can be extracted as arrays and stored alongside each corresponding time series row in the compact DataFrame.

Saving and loading files to disk

The fully merged DataFrame in its compact form takes up only ~10 MB. However, saving this file requires a little bit of engineering. If we try to save the file in the CSV format, it will not work because of the way we have stored arrays in pandas columns.

We can save it in pickle or parquet format, or any of the binary forms of file storage. This can work, depending on the size of the RAM available on our machines.

Although the fully merged DataFrame is just ~10MB, saving it in pickle format will make the size explode to ~15 GB.

What we can do is save this as a text file while making a few tweaks to accommodate the column names, column types, and other metadata that is required to read the file back into memory. The resulting file size on disk **still comes out to ~15 GB**, but since we are doing it as an I/O operation, we do not keep all that data in our memory. We call this the time series (.ts) format.

Saving and loading files to disk

If we don't need to store all of the DataFrame in a single file, you can split it into multiple chunks and save them individually in a binary format, such as parquet.

For our datasets, we will follow this route and split the whole DataFrame into chunks of blocks and save them as parquet files.

This is the best route for us for a few reasons:

- It leverages the compression that comes with the format
- It reads in parts of the whole data for quick iteration and experimentation
- The data types are retained between the read and write operations, leading to less ambiguity